

Programming Language Confusion: When Code LLMs Can't Keep their Languages Straight

Micheline Bénédicté Moumoula, Serge Lionel Nikiema
Abdoul Kader Kaboré, Jacques Klein, Tegawendé F. Bissyandé

University of Luxembourg, Luxembourg

{micheline.moumoula, lionel.nikiema, abdoulkader.kabore, jacques.klein, tegawende.bissyande}@uni.lu

Abstract—Large Language Models (LLMs) have achieved state-of-the-art performance across software engineering tasks, from code generation to translation. However, we identify and systematically evaluate a critical failure mode: Programming Language Confusion (PLC)—the generation of code in unintended languages despite explicit instructions. Through evaluation of 10 popular LLMs across six multilingual datasets (LiveCodeBench, BabelCode variants, HumanEval-XL, and McEval), we demonstrate that PLC is pervasive, with some specialized models exhibiting the highest confusion rates.

Our analysis reveals that PLC is not random noise but reflects systematic patterns: models consistently generate syntactically valid code even when it deviates from language specifications. This behavior produces distinct language migration patterns, most notably a strong default to Python and systematic shifts between syntactically similar language pairs (e.g., C#/Java). These migrations reflect statistical preferences learned from training data rather than goal-directed reasoning. We demonstrate that explicit language keywords provide the most effective mitigation, while natural language instructions have limited influence on model behavior. Furthermore, model quantization—though essential for practical deployment—significantly amplifies PLC and degrades syntactic stability in complex tasks.

Our findings underscore that language fidelity should be treated as a core evaluation dimension for code LLMs. We advocate for standardized benchmarks and prompt formats with explicit language constraints to enable more reliable assessment and foster the development of robust, multilingual code generation systems.

Index Terms—Code generation, Code translation, Code confusion, Large Language Model

I. INTRODUCTION

Beyond their original text generation function, Large Language Models (LLMs) now operate as sophisticated computational tools, achieving benchmark-leading performance across multiple technical domains, including language understanding, formal reasoning, code synthesis, and other diversely complex software engineering tasks [1]. In particular, code-specialized language models (a.k.a code LMs) have emerged as significant implementations of foundation models [2], [3], with substantial empirical evidence documenting their integration throughout the software development lifecycle [4], [5]. These models have been deployed across multiple development environments [6] and function within constrained IDE contexts [7], as automated programming aids [8], and as components in complex agent systems [9]. Studies have demonstrated their ability to automate repetitive coding tasks [10], [11], [12], [13], potentially reducing implementation time [14], augmenting

programmer efficacy [15], and expanding access to complex programming techniques for developers with varied skill levels [16].

As the adoption of AI-generated code in production environments accelerates, foundation models are presenting subtle reliability challenges that demand rigorous investigation [17]. Indeed, the integration of LLM-generated code into critical systems demands unprecedented levels of correctness, security, and robustness. Rigorous investigations into the phenomenon of “hallucination”—where models confidently produce non-functional or even software-unrelated outputs—have revealed concerning patterns that demand immediate attention. The research community has therefore developed diverse benchmarks to evaluate the functional accuracy and safety of code generation systems under realistic constraints [18], [19]. In parallel, researchers have constructed comprehensive taxonomies [20], [21] that categorize the wide spectrum of programming errors generated by Code LMs, ranging from basic syntax and logic issues to sophisticated security vulnerabilities and edge-case failures. These collective efforts underscore the critical challenge facing the field: how to leverage the productivity and innovation gains of LLM-driven automated code generation without compromising the quality, reliability, and security of software systems.

This paper. We investigate a surprising limitation of LLMs in the domain of software engineering: *their inability to consistently generate code in a user's desired programming language*, as exemplified in Listing 1. This limitation poses fundamental concerns beyond anecdotal failures since inadvertent confusion between two languages may introduce subtle, difficult-to-detect errors that compromise both reliability and security.

Our study investigates to what extent programming language confusion occurs across 10 LLMs on six datasets related to code generation and translation tasks, involving about 75k program samples written in 16 programming languages. A key finding of our study is that *programming language confusion in LLMs often reflects systematic output preferences rather than random mistakes*: models frequently migrate to languages in which they can produce more syntactically valid code. These migration patterns (for example, frequent moves toward Python and between syntactically similar language pairs) are consistent with statistical preferences in the learned model distribution, and should not be interpreted as evidence

Prompt:

```
class Solution {
    /**
     * Write a java function to find the
     *   ↪ last two digits in factorial of
     *   ↪ a given number.
     */
    public Integer lastTwoDigits(Integer n
    ↪ ) {
```

Output of CodeLlama-13B-Instruct-Q:

```
def get_last_two_digits_of_factorial(n):
    fact = 1
    for i in range(2, n+1):
        fact *= i
    return str(fact)[-2:]
# Test cases
assert get_last_two_digits_of_factorial(1)
    ↪ == "1"
assert get_last_two_digits_of_factorial
    ↪ (10) == "4"
assert get_last_two_digits_of_factorial
    ↪ (100) == "7"
```

Listing 1: Programming language confusion example

of deliberate or goal-directed behavior by the models.

Our contributions are as follows:

- We conduct the first comprehensive, large-scale empirical investigation of programming language confusion in LLMs, systematically analyzing code generation and translation tasks across multiple programming languages and natural languages.
- We quantify the impact of model quantization on PLC, demonstrating that this necessary deployment step significantly amplifies confusion and compromises the syntactic stability of model outputs.
- We provide novel insights into the factors contributing to programming language confusion, revealing critical limitations in current LLM design that affect multilingual code generation reliability.

II. BACKGROUND & RELATED WORK

This section contextualizes our investigation of programming language confusion within the broader landscape of code language models and reliability challenges in AI-assisted software engineering.

A. Code Language Models

Code language models have evolved rapidly since the introduction of early approaches such as code2vec [22] and CodeBERT [23]. Modern code LMs like Codex [18], AlphaCode [24], PaLM-Coder [2], and Code Llama [25] leverage large-scale transformer architectures trained on vast code repositories to generate functionally correct code from natural language specifications. These models typically employ similar pretraining objectives as general-purpose LLMs but with

specialized training data composed of code from various programming languages [8]. The effectiveness of code language models depends significantly on their representational capacity for programming languages. Unlike natural languages, programming languages exhibit strict syntactic constraints and semantic rules that must be followed precisely for code to execute correctly [26]. This presents unique challenges for language models, particularly in the context of multiple programming languages where syntax and paradigms can vary substantially [27]. Research by Xu et al. [8] demonstrates that these models can learn multiple programming languages simultaneously, but the mechanisms governing cross-language interaction and potential interference remain underexplored.

B. Reliability Challenges in LLM-Generated Code

As LLM-generated code transitions from research settings to production environments, reliability has emerged as a critical concern. Recent studies have documented various shortcomings in code generation capabilities, including syntactic errors [20], semantic inconsistencies [21], security vulnerabilities [28], and hallucinated API references [29]. Taxonomies of errors in LLM-generated code have been proposed by Liu et al.[20] and Pan et al.[21], categorizing issues from basic syntax errors to complex logical flaws. However, these taxonomies do not specifically address programming language confusion as a distinct category of error. The economic impact of such errors has been analyzed by Sobania et al. [30], who found that fixing AI-generated code defects can sometimes exceed the effort saved by the initial code generation. Various benchmarks have been developed to evaluate code LMs, including HumanEval [18], MBPP [31], and CodeXGLUE [32], which test functional correctness across different programming tasks. More recently, multi-language benchmarks like MultiPL-E [33] have enabled cross-linguistic comparisons, but they typically evaluate each language in isolation rather than examining cross-language interference explicitly.

C. Cross-language Phenomena in Language Models

While extensive research exists on multilingual capabilities in natural language models [34], similar investigations in programming languages remain limited. The phenomenon of code-switching and language mixing in natural language has been studied by Winata et al. [35], who found that language models can struggle with maintaining linguistic boundaries during generation. Analogous behaviors in code generation contexts would have significant implications for software reliability. Cross-lingual transfer learning has been explored by Ahmad et al.[27] and Wang et al.[36], demonstrating that models can leverage syntactic similarities between programming languages. However, these studies focus primarily on leveraging cross-language knowledge for improved performance **rather than investigating potential negative interference** between languages.

D. Human Programming Language Cognition

Research in human cognition provides relevant context for understanding language confusion in LLMs. Studies by

Kahneman [37] on dual-system thinking and cognitive load theory applied to programming by Siegmund et al. [38] suggest that context switching between programming languages imposes significant cognitive demands. Human programmers develop explicit strategies to manage these demands and avoid confusion between languages with similar syntax [39]. Peitek et al. [40] have used fMRI studies to demonstrate that different programming languages activate distinct neural patterns in human programmers, suggesting specialized cognitive processes for language-specific comprehension. Whether LLMs exhibit analogous specialization or more fluid boundaries between language representations remains an open question.

E. Position of Our Work

Our research differs from previous work in several key respects. Unlike studies that evaluate models on individual programming languages in isolation [18], [31], we specifically investigate cross-language confusion and interference. While taxonomies of LLM coding errors [20], [21] provide useful frameworks, they do not specifically address language confusion as a distinct error category—a gap our work aims to fill. Most closely related to our research is work by Cassano et al. [33] on multi-language evaluation, but their focus remains on comparative performance rather than systematic analysis of cross-language confusion patterns. Similarly, while Ahmad et al. [27] explore cross-lingual transfer between programming languages, they do not investigate negative transfer or confusion. Our systematic evaluation across multiple languages and models, coupled with our analysis of contributing factors and proposed mitigation strategies, represents a novel contribution to understanding the reliability challenges of LLM-based code generation in multilingual contexts.

III. EXPERIMENTAL SETUP

This section details our experimental methodology for investigating programming language confusion in LLMs, including our research questions, model selection criteria, datasets, evaluation metrics, and procedural workflow.

A. Research Questions

Our investigation focuses on three main research questions.

- **RQ1:** *To what extent does programming language confusion manifest in LLM-based code generation?* We quantify the prevalence of language confusion in code generation tasks and analyze its correlation with code correctness across different programming languages and model architectures.
- **RQ2:** *What factors influence programming language confusion during LLM-based code translation?* We investigate whether LLMs maintain target language adherence when explicitly tasked with translating code between programming languages, and examine how translation direction and language pairs affect confusion patterns.
- **RQ3:** *How does model quantization affect programming language confusion in LLM-based code tasks?* We evaluate

whether quantized models exhibit different levels of language confusion or code correctness compared to their full-precision counterparts across the generation and translation settings.

B. Models

We evaluate 10 LLMs spanning both general-purpose and code-specialized architectures to enable comparative analysis of language confusion tendencies. Our selection includes:

- **General-purpose LLMs:** DeepSeek-V2-Lite-Instruct-16B [41], GPT-3.5-Turbo [42], GPT-4.1-Mini [43], LLaMA3.1-8B-instruct [44], Mistral-7B-Instruct [45], Qwen2.5-Instruct-14B [46]
- **Code-specialized LLMs:** CodeLLaMA-13B-Instruct [25], DeepSeek-Coder-V2-Lite-Instruct-16B [41], Qwen2.5-Coder-Instruct-14B [46], StarCoder2-15B-Instruct [47].

Model selection criteria include architectural diversity, parameter scale variation (7B to 16B parameters), demonstrated performance on coding benchmarks [8], [13]. For all experiments we used greedy decoding (temperature $T = 0$) to mitigate randomness while preserving output coherence, following established practices in the code-generation literature [29]. To elicit full, self-contained solutions, we appended a small instruction prefix to the original benchmark prompt (see Appendix).

C. Datasets

To comprehensively assess language confusion across diverse contexts, we selected established multilingual datasets for both code generation and translation tasks:

- **Code Generation Datasets:**
 - **LiveCodeBench** [48]: A holistic, contamination-free benchmark using time-tagged competitive programming problems to evaluate code LLMs across generation, debugging, and execution tasks.
 - **BabelCode-HumanEval** [49]: A cross-lingual extension of the HumanEval benchmark [18] spanning 16 programming languages, with consistent function signatures, documentation standards, and test cases across languages.
 - **BabelCode-MBPP**: A multilingual adaptation of the MBPP dataset [19] containing approximately 1,000 basic programming problems with natural language descriptions and test cases.
 - **BabelCode-TP3**: Derived from Python Programming Puzzles [50], this dataset features verification functions of varying complexity, from string manipulation to advanced algorithms.
 - **HumanEval-XL** [51]: An extension of HumanEval with problem statements translated to multiple natural languages, enabling evaluation of cross-linguistic influence on code generation.
- **Code Translation Dataset:**
 - **McEval** [52]: A comprehensive benchmark for evaluating code generation and understanding, containing diverse programming tasks and problem descriptions, supporting

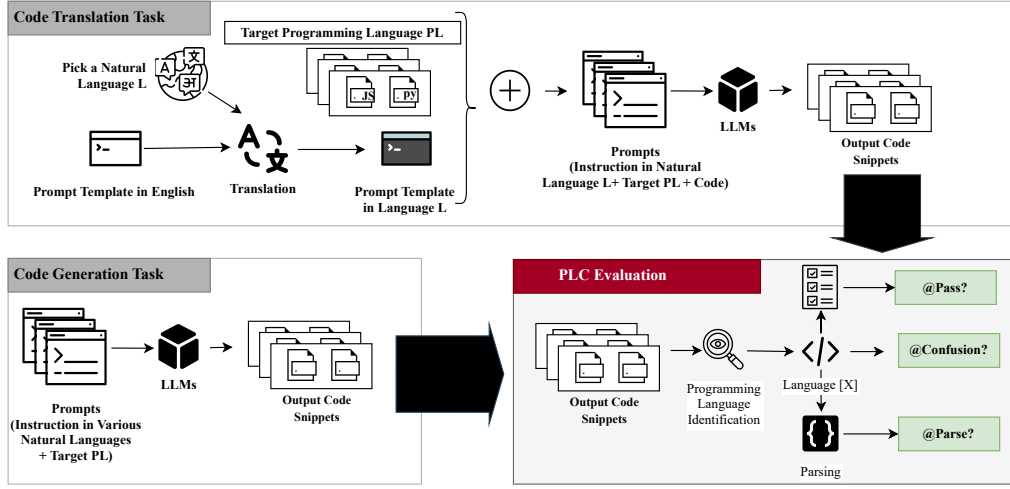


Fig. 1: Experimental workflow for programming language evaluation

structured assessment of translation capabilities between programming language pairs.

TABLE I: Summary information on experimental datasets

Task	Instruction language	Dataset name	Languages	# of samples
Code generation	English	LiveCodeBench	1 PL	880
		BC-TP3	13 PLs	5920
		BC-MBPP	13 PLs	13673
		BC-HumanEval	13 PLs	2576
	23 NLs	HumanEval-XL	12 PLs	22080
Code translation	23 NLs	McEval	14 PLs	33488

Dataset selection was guided by three criteria: (1) coverage of diverse programming paradigms and language families, (2) availability of parallel implementations across languages, and (3) established usage in prior code generation research [8], [29], [18].

D. Evaluation Metrics

We define three complementary metrics to evaluate the extent of language confusion:

- **Language Confusion Pass Rate (LCPR):** Measures the proportion of the generated code samples that correctly match the target programming language. Formally defined as:

$$\text{LCPR} = \frac{\text{Number of samples in correct language}}{\text{Total number of samples}} \quad (1)$$

This metric, which builds on precision, quantifies a model's ability to maintain language boundaries during generation, i.e. whether it can accurately generate code in the expected programming language. The determination of the language, given a code, is done following the methodology described in Section III-E.

- **Code Parsing Pass Rate (CPPR):** Assesses the syntactic validity of generated code by measuring the proportion of samples that parse without errors in the detected language:

$$\text{CPPR} = \frac{\text{Number of samples that parse successfully}}{\text{Total number of samples}} \quad (2)$$

- **Dominant Migration Rate (DMR):** Measures the proportion of language-confused samples that migrate to a specific target language (e.g., Python):

$$\text{DMR}(X) = \frac{\text{Number of confused samples in language } X}{\text{Total number confused samples}} \quad (3)$$

These metrics are based on well-established principles from compiler theory and multilingual NLP evaluation [34], tailored specifically to analyze programming language confusion.

E. Methodology

Our experimental workflow, illustrated in Figure 1, consists of two primary evaluation streams for code generation and translation tasks:

- **Code Generation Task:** We employ a three-stage process:
 - 1) *Language Model Inference:* Each model generates code responses to prompts from the five code generation benchmarks.
 - 2) *Programming Language Detection:* We implement an ensemble approach using three specialized tools to reliably identify the programming language of generated code:
 - `Highlight.js`¹: A widely-used syntax detection library supporting 192 programming languages.
 - `Guesslang`²: A deep learning-based language identification system with > 90% reported accuracy across 50+ languages.
 - `Philomath-1209`³: A fine-tuned CodeBERTa model specifically designed for programming language identification.
 - `PLangRec` [53]: A deep-learning model for programming language recognition that leverages a large balanced dataset and ensemble learning to classify

¹<https://highlightjs.org/>

²<https://guesslang.readthedocs.io/>

³<https://huggingface.co/philomath-1209/programming-language-identification>

code at line, snippet, and file level with state-of-the-art accuracy.

- 3) *Language Determination Algorithm*: We employ a weighted consensus approach prioritizing Philomath-1209 due to its demonstrated accuracy. When tools disagree, we apply a decision tree that considers language-specific characteristics and the confidence scores of each detector. We validated the pipeline on a subset covering the 16 programming languages used in our study and found the ensemble achieves an overall accuracy of 98.2%, confirming the reliability of our detection approach; per-language results and full confusion matrices are provided in the supplementary file.
- 4) *Evaluation*: The extracted code is then parsed using Tree-sitter⁴, an incremental parsing library that generates concrete syntax trees. This allows us to identify syntax errors within the code. Finally, we compare the target programming language with the detected programming language to identify the language confusion, where code is mistakenly written in the wrong language.

• **Code Translation Task**: We extend our methodology to translation contexts:

- 1) *Instruction Translation*: Prompt instructions were translated into 23 natural languages using GPT-4 and validated through back-translation to ensure semantic fidelity.
- 2) *Translation Generation*: LLMs translate source code between language pairs while following instructions in various natural languages.
- 3) *Language Detection and Validation*: We apply the same detection ensemble and Tree-sitter parsing as in the generation task to identify language confusion instances.

This methodology enables systematic identification and quantification of programming language confusion across diverse contexts, providing the empirical foundation for addressing our research questions.

IV. EVALUATION

This section presents our empirical findings organized by research question. For each research question defined in Section III-A, we first describe the corresponding experiments conducted and then, present quantitative results, discuss and analyze key implications.

A. [RQ1] Programming Language Confusion in LLM-Based Code Generation

Goal. We investigate how effectively LLMs maintain programming language fidelity during code generation. Specifically, we quantify the extent of PLC and examine factors that influence it, including language families, model specialization, and instruction language.

Experiments. We conducted three sets of experiments:

- 1) An evaluation on LiveCodeBench using English instructions that omit explicit language specification. This determines the models’ default language schema (baseline PLC).

- 2) An evaluation on BabelCode datasets with explicit English instructions, assessing LCPR, CPPR, and DMR across 13 diverse target languages.
- 3) An analysis using HumanEval-XL with instructions in 23 natural languages to examine how instruction language affects LCPR.

We also report in this section the Functional Pass Rate (FPR) that measures the proportion of generated programs that execute successfully and pass at least one unit test:

TABLE II: PLC during Code Generation with LiveCodeBench

LLMs	LCPR	CPPR non confuse %	CPPR confuse %	DMR %
CodeLLama-13B-Instruct	98.86	99.77	100.00	100 (c++)
DeepSeek-Coder-V2-Lite-Instruct-16B	100.00	99.89	-	-
DeepSeek-V2-Lite-Instruct-16B	99.89	99.54	0.00	100 (scala)
GPT-3.5-Turbo	100.00	99.66	-	-
GPT-4.1-Mini	99.89	100.00	0.00	100 (scala)
LLama3.1-8B-Instruct	97.22	91.79	87.50	91.67 (c++)
Mistral-7B-Instruct	66.70	99.66	93.10	93.10 (c++)
Qwen2.5-Instruct-14B	99.89	99.89	0.00	100 (scala)
Qwen2.5-Coder-Instruct-14B	100.00	100.00	-	-
StarCoder2-15B-Instruct	100.00	99.89	-	-

Results.

1) *Python as the Universal Attractor*: Table II presents PLC results on LiveCodeBench, a Python-focused benchmark where instructions deliberately omit explicit language specification. The results demonstrate that most LLMs have internalized a strong Python default, with four models achieving a perfect 100.00% LCPR. This confirms Python as the established default coding language when instructions omit explicit language specification. However, this fidelity is not universal, with Mistral-7B-Instruct exhibiting significant PLC (66.70% LCPR), often defaulting to C++. A critical finding is the instability in code quality independent of language confusion, evidenced by Llama3.1-8B-Instruct showing reduced syntactic correctness (91.79% $CPPR_{non-confused}$) even when correctly targeting Python, suggesting a fundamental failure in syntactic generation beyond mere language switching.

2) *Quantifying PLC (BabelCode)*: From Table III, we observe significant LCPR variation. Qwen2.5-Instruct-14B demonstrates the strongest language adherence at 97.60% LCPR, while StarCoder2-15B-Instruct exhibits the lowest fidelity at 74.33% LCPR. Crucially, some general-purpose models (e.g., GPT-3.5-Turbo) rival or outperform code-specialized models (e.g., CodeLLama-13B-Instruct), supporting the finding that code specialization does not guarantee superior language boundary maintenance.

Systematic Language Migration (CPPR). A striking pattern emerges: syntactic correctness is preserved despite language confusion. Most models achieve $CPPR_{non-confused}$ above 95%. Intriguingly, $CPPR_{confused}$ values are often comparable to the non-confused rates. This counterintuitive behavior suggests a systematic language migration where models prioritize structural validity, deliberately defaulting to languages (likely those over-represented in training) where they are more confident in producing syntactically reliable code, even when this contradicts the language request.

Functional Degradation. Supplementary file Table 2 reveals that high CPPR is a false indicator of quality. While confused

⁴<https://tree-sitter.github.io/tree-sitter/>

TABLE III: PLC in % during Code Generation with BabelCode (BC)

LLMs	Dataset	LCPR	CPPR non-confuse	CPPR confuse	DMR %
CodeLLama-13B-Instruct	BC-TP3	88.18	98.64	86.47	43.98 (java)
	BC-MBPP	87.26	97.35	95.84	36.01 (java)
	BC-HumanEval	82.71	97.85	91.14	38.00 (py)
	All	86.96	97.75	92.89	37.09 (java)
DeepSeek-Coder-V2-Lite-Instruct-16B	BC-TP3	77.39	99.17	85.77	91.89 (py)
	BC-MBPP	91.63	99.73	97.92	62.83 (py)
	BC-HumanEval	84.89	99.83	100.00	70.68 (py)
	All	87.06	99.61	92.54	77.41 (py)
DeepSeek-V2-Lite-Instruct-16B	BC-TP3	85.23	97.92	91.07	42.75 (py)
	BC-MBPP	86.63	98.77	95.34	54.60 (py)
	BC-HumanEval	84.36	98.14	95.61	46.08 (py)
	All	86.00	98.48	94.19	50.20 (py)
GPT-3.5-Turbo	BC-TP3	98.74	99.10	71.67	76.67 (java)
	BC-MBPP	91.73	99.56	97.34	54.94 (java)
	BC-HumanEval	92.48	98.94	98.05	66.23 (java)
	All	93.69	99.36	96.05	57.67 (java)
GPT-4.1-Mini	BC-TP3	97.65	99.28	78.38	31.53 (java)
	BC-MBPP	92.48	99.87	96.70	43.59 (py)
	BC-HumanEval	94.70	99.95	97.22	80.56 (java)
	All	94.12	99.72	94.80	45.38 (java)
LLama3.1-8B-instruct	BC-TP3	72.28	95.13	91.23	40.56 (js)
	BC-MBPP	81.51	96.42	95.62	42.35 (py)
	BC-HumanEval	67.78	94.64	95.06	36.27 (js)
	All	77.47	95.92	94.10	37.08 (py)
Mistral-7B-Instruct	BC-TP3	93.10	98.24	95.31	87.50 (js)
	BC-MBPP	92.55	99.33	96.67	39.21 (java)
	BC-HumanEval	87.03	98.80	96.17	55.17 (js)
	All	92.07	98.98	96.26	46.34 (js)
Qwen-2.5-14B-Instruct	BC-TP3	99.20	99.42	92.11	65.79 (js)
	BC-MBPP	96.93	99.54	98.51	69.85 (java)
	BC-HumanEval	97.51	99.75	98.04	62.75 (java)
	All	97.60	99.53	97.88	63.92 (java)
Qwen-2.5-Coder-14B-Instruct	BC-TP3	95.30	99.71	99.02	50.20 (py)
	BC-MBPP	95.30	99.71	99.02	50.20 (py)
	BC-HumanEval	97.51	99.80	98.04	43.14 (java)
	All	96.56	99.77	99.01	50.08 (py)
StarCoder2-15B-Instruct	BC-TP3	65.24	99.12	93.10	78.92 (py)
	BC-MBPP	78.74	99.40	96.86	56.64 (py)
	BC-HumanEval	71.58	99.22	98.21	71.91 (py)
	All	74.33	99.32	95.68	66.59 (py)

outputs maintain high CPPR, their FPR degrades substantially. For instance, StarCoder2-15B-Instruct’s FPR drops from 39–76% when correct to 21–63% when confused. This shows that PLC introduces semantic errors beyond syntax—models generate code that is parseable but fails unit tests, revealing that systematic language selection optimizes for surface-level correctness at the expense of functional accuracy. However, the fact that models still produce some functionally correct code in confused samples suggests that cross-language semantic understanding is partially preserved despite language boundary violations.

3) *Language Migration Patterns and Biases*: We visualize the directional analysis of language migration in Figure 2. These chord diagrams reveal how PLC occurs, highlighting distinct migration patterns:

- **Python as Universal Attractor**: Python serves as the dominant destination language (high DMR) across confused models, reinforcing its status as the primary default schema.
- **Family-Based Confusion**: Consistent migration occurs between structurally similar languages, such as C#/C++ and Java, and PHP/TypeScript shifting to JavaScript.
- **Diffuse Migration**: Languages like Lua, Kotlin, and Scala exhibit highly diffuse migration patterns, suggesting they are the least tethered to distinct internal representations and thus the most prone to incorrect generation.

Models with higher LCPR exhibit fewer and thinner migration ribbons, indicating more selective confusion, while low-LCPR models show widespread, distributed confusion, suggesting a

failure to maintain clear internal language boundaries.

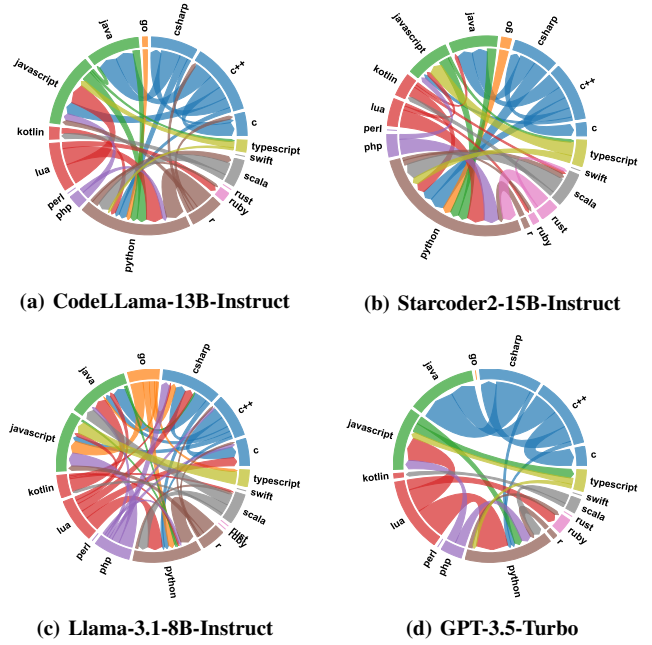


Fig. 2: Migration Patterns in Programming Language Confusion during Code Generation (with BabelCode dataset). **NOTE**: Data for all models are provided in the Supplementary file.

4) *Impact of Natural Language and Explicit Specification*: The HumanEval-XL results (Table IV), derived from prompts containing explicit language keywords, show a dramatic improvement in fidelity compared to the multilingual evaluation on BabelCode. This demonstrates that explicit language specification is highly effective at constraining model output and mitigating the general PLC observed previously. However, two secondary findings are established. When the target language is explicitly specified, nearly all models achieve $LCPR > 99.00\%$, with several reaching 100.00% . This confirms that their core competency to adhere to explicit instructions is robust. Despite instructions spanning 23 different natural languages, the variation between the “English” and “Non-English” LCPR is negligible (generally $< 3\%$). This reinforces the conclusion that the model’s tendency to confuse programming languages stems from deeper architectural or training data limitations, rather than difficulties interpreting the natural language of the prompt.

Overall, our analysis reveals six key findings:

- **Finding 1.1: PLC is pervasive without explicit cues, regardless of specialization.** Even code-specialized LLMs show significant confusion on the multi-lingual BabelCode task. Surprisingly, specialization does not guarantee superior language fidelity.
- **Finding 1.2: Syntactic correctness is preserved despite language confusion.** LLMs prioritize structural validity, leading to a systematic language migration towards syntactically familiar languages (e.g., Python, JavaScript) when uncertain.

- **Finding 1.3: High CPPR masks critical functional failure, yet some correctness remains.** While language confusion introduces semantic errors, leading to a substantial degradation in FPR, models still produce code that passes at least one unit test in confused samples. This suggests that the confusion does not always result in a total functional failure.
- **Finding 1.4: Strong directional biases exist in language confusion patterns.** Models predominantly migrate code to Python and between syntactically similar language families.
- **Finding 1.5: Explicit language keywords are the most effective mitigation for PLC.** Introducing explicit programming language keywords into the prompt significantly improves LCPR across all models.
- **Finding 1.6: Natural language instruction variations have minimal impact on PLC.** The instruction’s natural language (e.g., Spanish vs. English) does not drive the confusion; the issue is the internal instability of language boundaries.
- **Finding 1.7: Code specialization does not guarantee language fidelity.** Specialized code models such as StarCoder2-15B-Instruct, DeepSeek-Coder-V2-Lite-Instruct-16B, and CodeLLama-13B-Instruct—despite superior code generation capabilities—exhibit higher PLC rates than smaller general-purpose models like Mistral-7B-Instruct. This suggests that code quality and language fidelity may require different optimization strategies.

[RQ-1] *LLMs consistently confuse programming languages without explicit specification, exhibiting strong biases toward Python and between syntactically similar languages. While explicit language keywords drastically reduce confusion, the choice of natural instruction language has a negligible effect. Though syntactic correctness is preserved (systematic migration), this masks a severe degradation in functional correctness, even if some confused samples still pass minimal tests. Model size does not predict language fidelity, as specialized code models often show higher confusion rates than smaller general-purpose models.*

B. [RQ2] Programming Language Confusion in LLM-Based Code Translation

Goal. We investigate how programming language confusion manifests during code translation tasks, examining whether models maintain target language adherence when explicitly tasked with converting code between programming languages. **Experiments.** We evaluated all models on the McEval dataset, measuring the language fidelity rate of the translated code. We analyzed performance variations across different source and target language pairs.

Results. Table V presents the overall results.

1) *Translation Task Mitigates General PLC:* A primary observation is the significantly higher overall LCPR observed in the translation task compared to the multilingual code generation task. In Section IV-A, average LCPR ranged from

74.33% to 97.60%; in RQ2, LCPR is consistently high, ranging from 98.22% to 99.73% (Table V).

This result indicates that the explicit requirement to handle two distinct programming languages (source and target) forces the LLMs into a different, more constrained operational mode, drastically reducing the propensity for the catastrophic, non-adherence errors seen in simple generation. The strong, explicit language tags in the translation prompt (e.g., “Translate this code to Python”) appear to serve as a mandatory constraint, mitigating the implicit language biases that emerged when the target language was the only instruction + function signature.

2) *High Fidelity is Not Universal:* While overall LCPR is high, we observe performance plateaus and surprising rankings:

- **Generalist Models Excel:** GPT-3.5-Turbo and GPT-4.1-Mini achieve nearly perfect fidelity, suggesting that strong general instruction-following capabilities are paramount in this two-language, explicitly constrained setting.
- **Specialization Does Not Guarantee Control:** Models highly specialized in code, such as CodeLLaMA-13B-Instruct (99.02% LCPR) and StarCoder2-15B-Instruct (98.22% LCPR), lag behind models like Qwen2.5-Coder-Instruct (99.73% LCPR) and even the general-purpose GPT models. This reinforces the finding from RQ1 that code specialization does not necessarily translate to superior control over language boundaries or instruction adherence.
- **The Worst-Case Scenario is Rare:** The PLC observed in code translation is minimal (less than 2% for most models). This suggests that confusion in translation is not a failure of identifying the target language, but likely a failure to escape the source language schema or a fallback to a dominant language when translation itself fails (i.e., the model outputs the source language or a dominant language like Python/C++ instead of the target).

3) *Minimal Impact of Natural Language:* The analysis of language fidelity based on the natural language of the surrounding instruction further confirms the finding from RQ1. The difference in LCPR between English and Non-English instructions is minimal across all models (generally < 0.4%). This confirms that in highly explicit, structured coding tasks like translation, the model’s fidelity is driven by the programming language tokens and syntax in the prompt.

Overall, our analysis reveals four key findings:

- **Finding 2.1: Explicit Translation Mitigates General PLC.** The highly constrained nature of the code translation task drastically reduces the wide-ranging language confusion observed during general code generation. The requirement to process a source language acts as a strong anchor, constraining the model’s output to the explicit target language.
- **Finding 2.2: Translation Fidelity is Robust (but not perfect).** LCPR is consistently high (> 98%) for the translation task. When confusion does occur, it is a rare failure mode, likely rooted in the model’s inability to fully switch from

TABLE IV: Language Confusion Pass Rates on HumanEval-xl prompts (Data for all the Natural Languages table is available in the Supplementary file)

Language	CodeLLaMA 13B-Instruct	DeepSeek-Coder-V2 Lite-Instruct-16B	DeepSeek-V2-Lite Instruct-16B	GPT-3.5 Turbo	GPT-4.1 Mini	LLaMA-3.1-8B Instruct	Mistral-7B Instruct	Qwen2.5 Instruct-14B	Qwen2.5-Coder Instruct-14B	StarCoder2 15B-Instruct
English	96.10	99.79	99.36	100.00	99.89	99.89	99.79	100.00	100.00	95.53
Non-English	95.96	99.54	98.90	99.82	99.98	99.50	99.45	99.98	99.99	92.96
All	95.97	99.55	98.92	99.83	99.97	99.51	99.47	99.98	99.99	93.07

the source language schema or a total failure leading to a default language fallback.

- **Finding 2.3: Specialization is not a Translation Advantage.** General-purpose models (GPTs variants) often achieve higher translation fidelity than larger, code-specialized models, confirming that adherence to complex, two-part instructions (source $\rightarrow$ target) is more about general robustness than code training volume alone.
- **Finding 2.4: Natural Language in Translation.** As with code generation, the natural language of the instruction has a negligible impact on translation fidelity. The model is constrained primarily by the explicit programming language keywords.

[RQ-2] Programming Language Confusion is minimal in the explicit code translation task, with LCPR consistently above 98% across all models. The need to process a source language drastically mitigates the PLC observed in general generation. However, some specialized models still fail more often than generalist ones, confirming that explicit constraints and general instruction adherence are the dominant factors, not code specialization.

C. [RQ3] How does model quantization affect programming language confusion in LLM-based code tasks?

Goal. We investigate how model quantization impacts the ability of LLMs to maintain programming language fidelity during code translation. We quantify PLC in quantized models and compare it to their full-precision counterparts to understand the effects of reduced numerical precision on language preservation and code correctness.

Motivation for Quantization Comparison. Quantized models (e.g., Q4_K_M or Q5_K_M via frameworks like Ollama⁵) are crucial for practical, local deployment and edge computing of LLMs. We aim to determine the real-world fidelity penalty incurred by reducing model precision for accessibility. Comparing these Quantized (Q) models against their Full-Precision (FP) counterparts evaluated in RQ1 and RQ2 allows us to isolate the degradation specific to the quantization process.

Experiments. We conduct code generation and translation experiments using all datasets from RQ1 and RQ2 for a subset of models, focusing on CodeLLaMA-13B-Instruct, LLaMA3.1-8B-Instruct, Mistral-7B-Instruct, and StarCoder2-15B-Instruct, obtained from the Ollama framework. Quantized versions (Table VI) were evaluated alongside their full-precision counterparts (Tables III and V) to assess the difference in PLC, syntactic correctness, and language migration patterns.

Results. Our analysis reveals several key patterns.

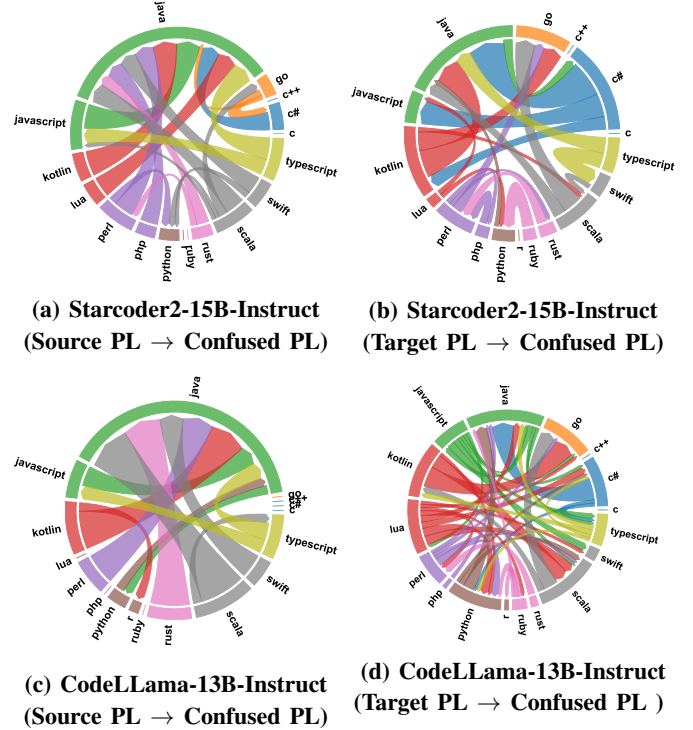


Fig. 3: Directional Analysis of Programming Language Confusion in Translation Tasks.

NOTE: Data for all models are provided in the Supplementary file.

1) *Quantization Significantly Increases PLC:* Comparing the LCPR values for the BabelCode dataset reveals a substantial drop in language fidelity for the quantized models, confirming the findings from the previous experiments. CodeLLaMA-13B-Instruct’s LCPR drops from $\sim 87.0\%$ (FP) to 73.57% (Q). Similarly, StarCoder2-15B-Instruct’s LCPR drops dramatically from $\sim 74.3\%$ (FP) to 51.87% (Q). This reduction in fidelity is also evident in the HumanEval-XL dataset. For StarCoder2-15B-Instruct, the LCPR drops from $\sim 93.07\%$ to 68.93% (Q). This indicates that the information crucial for maintaining explicit constraints is also sensitive to numerical precision loss. Quantization effectively makes models “forget” the finer distinctions between languages, amplifying the intrinsic confusion tendencies found in RQ1, even when explicit guidance is given.

2) *Quantization Impacts Default Bias and Migration Patterns:* The analysis of the Figure 4 and DMR values reveals how quantization affects the models’ internal language schemas:

- **Shift in Default Language:** For the generation task with no explicit language (LiveCodeBench), the quantization process

⁵<https://ollama.com/>

TABLE V: Language Confusion Pass Rates by Natural Languages during Code Translation with McEval

Language	CodeLLaMA 13B-Instruct	DeepSeek-Coder-V2 Lite-Instruct-16B	DeepSeek-V2-Lite Instruct-16B	GPT-3.5 Turbo	GPT-4.1 Mini	LLaMA-3.1-8B Instruct	Mistral-7B Instruct	Qwen2.5 Instruct-14B	Qwen2.5-Coder Instruct-14B	StarCoder2 15B-Instruct
English	99.01	99.79	98.80	99.65	99.23	99.37	99.93	99.37	99.65	98.59
Non-English	99.02	99.60	98.76	99.61	99.31	99.22	99.43	99.44	99.74	98.20
All	99.02	99.61	98.76	99.61	99.30	99.22	99.45	99.43	99.73	98.22

can shift the model’s default bias. The FP models generally defaulted to Python (RQ1), but the Q version of LLaMA3.1-8B-Instruct now exhibits a strong bias towards `C++` (89.55% DMR). This shows that quantization does not just weaken boundaries; it can rearrange the internal language hierarchy.

- **Diffuse Migration:** Consistent with the qualitative analysis of the chord diagrams (See Appendix), the DMR values for BabelCode are slightly lower for the Q models (e.g., Mistral-7B-Instruct drops from $\sim 54\%$ to 41.43%). This suggests a qualitative change in confusion: instead of simply falling back to the strongest learned bias (Python), the model’s output is scattered to a more diverse and diffuse array of incorrect languages, as precision loss weakens the target representations.

Overall, our analysis reveals four key findings:

- **Finding 3.1: Quantization is a significant amplifier of PLC.** Reducing model precision drastically increases PLC across all tasks, with LCPR dropping by up to 20 percentage points, even when explicit language constraints are given.
- **Finding 3.2: Precision Loss Weakens and Re-arranges Language Boundaries.** Quantization can cause the model to shift its default language bias and lead to a more diverse and diffuse migration pattern.
- **Finding 3.3: Quantization Reduces Functional Reliability.** Quantization lowers functional pass rates across both confused and non-confused samples (Tables 2 and 3 in the Appendix).
- **Finding 3.4: Practical LLM Deployment Trades Off Fidelity.** The necessary process of quantization for local deployment introduces a significant, quantifiable penalty on language fidelity and reliability, confirming that the high performance reported on FP models may not translate directly to practical, resource-constrained use cases.

[RQ-3] *Model quantization significantly amplifies PLC across all tasks, with LCPR dropping substantially even when explicit constraints are provided. The loss of precision also causes a breakdown of syntactic stability, leading to a simultaneous failure of language fidelity and code correctness in confused outputs during the complex translation task.*

V. DISCUSSION

This section examines the implications of our findings and discusses limitations of our study.

A. Key Insights

Our study shows that language confusion in LLMs is complex and depends heavily on how the model is built, the task it’s doing, and how it’s deployed.

TABLE VI: PLC in % using quantized models from Ollama

LLMs	Dataset	LCPR	CPPR non-confuse	CPPR confuse	DMR %
CodeLLama-13B-Instruct-Q	LiveCodeBench	97.49	99.88	81.82	40.91 (c++)
	BabelCode	73.57	98.20	96.37	71.03 (py)
	HumanEval-XL	84.15	88.50	97.61	94.00 (py)
	McEval	98.91	81.91	63.01	30.06 (java)
LLama3.1-8B-instruct-Q	LiveCodeBench	92.36	98.89	91.04	89.55 (c++)
	BabelCode	65.08	98.89	98.03	75.51 (py)
	HumanEval-XL	98.72	88.52	94.87	91.21 (py)
	McEval	99.67	81.46	55.05	72.48 (java)
Mistral-7B-Instruct-Q	LiveCodeBench	99.89	97.72	100.00	100.00 (c++)
	BabelCode	86.05	97.71	97.57	41.43 (js)
	HumanEval-XL	99.75	88.35	88.89	38.89 (js)
	McEval	99.68	80.14	35.24	40.95 (js)
StarCoder2-15B-Instruct-Q	LiveCodeBench	99.66	99.66	100.00	66.67 (java))
	BabelCode	51.87	99.31	98.87	86.95 (py))
	HumanEval-XL	68.93	92.40	99.36	94.77 (py)
	McEval	98.14	85.51	50.00	66.45 (java)

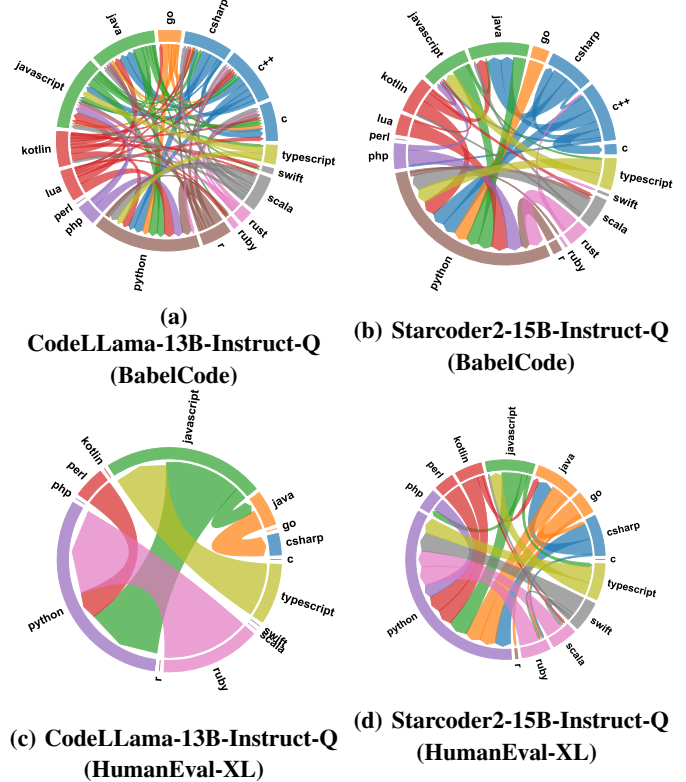


Fig. 4: Directional Analysis of Programming Language Confusion when using quantized models

Smart Error: Prioritizing Syntax over Language. When an LLM gets confused about which language to use, the error isn’t random. It’s a systematic move where the model tries to keep the code syntactically correct (meaning the code looks valid) even if it’s in the wrong language. This is supported by high rates of correct-looking, but wrong-language, code. When this confusion happens, models most often default to Python, which acts like a “Universal Attractor.” This fallback happens because Python is so common in the model’s training data.

This shows the model is stuck between two goals: following the language instruction and producing code that simply “looks right.”

The Hidden Cost of Using Smaller Models (Quantization Penalty). To run LLMs locally or on small devices, they are often quantized (simplified by reducing numerical precision). This process significantly increases language confusion. Reducing the model’s precision weakens its memory of distinct programming languages, leading to two major problems: (1) a noticeable drop in how well the model sticks to the target language, and (2) in complex tasks like code translation, a complete failure of the output code’s structure and syntax when it gets confused. This means the impressive scores of large, full-size models might not hold up when they are shrunk for real-world use.

The Prompt’s Natural Language Doesn’t Matter. Our analysis across multiple spoken languages shows that the language of the instruction (e.g., English, Spanish) has very little effect on whether the model gets confused about the programming language. The confusion comes from the model’s internal technical knowledge of programming languages, not its ability to understand the prompt’s natural wording. Therefore, efforts to improve reliability should focus on the technical details of the prompt, not its grammar or wording.

B. Mitigation Strategies

Based on these findings, we recommend three main ways to reduce Programming Language Confusion:

Technical and Redundant Prompt Design. Since explicit programming language keywords are the best way to prevent confusion, and this prevention is easily lost during quantization, prompts must be extra detailed. Use redundant and highly technical cues like including language-specific boilerplate, function definitions, and using specific formatting tags (e.g., ````python`) to act as strong boundaries for the model.

Test Quantized Models for Confusion. When choosing models for local or small-device deployment, don’t just look at the performance of the full-size version. Because model behavior changes so much after being quantized, the final, reduced-size version must be thoroughly tested for its robustness against language confusion. Since simpler models can sometimes be more reliable than specialized ones in certain constrained tasks, a careful, task-specific evaluation of the quantized model is essential before deployment.

C. Threats to Validity

Several limitations should be considered when interpreting our results, along with the steps we took to mitigate them.

Model Selection and Generalizability. Our study evaluated a selection of LLMs with varying architectures and sizes. While this provides a good range of current models, the field is evolving very quickly, meaning our findings might not apply perfectly to every new model released.

Language Identification Accuracy. Our method for identifying the programming language relies on a multi-tool approach, combining four different tools. Although this setup greatly

improves reliability, it is not perfect. To reduce the chance of misclassification, we used a weighted consensus algorithm that trusts the most accurate tool while getting validation from the others.

Limited Language and Task Scope. While our study covered 16 programming languages and 23 natural languages across different task types (generation and translation), we did not test every possible language or task. We managed this limitation by choosing languages that represent diverse programming styles (like imperative, functional, and object-oriented) and included both widely-used and less common languages to ensure broad coverage within our testing limits.

Intra-Statement Confusion Detection. We found cases where models mixed the syntax of multiple languages within a single line or code block. Since current tools cannot formally evaluate this very fine-grained confusion, our analysis focused on identifying overall syntactic errors. These errors, often resulting from mixing language constructs, partially helped us understand where this fine-grained confusion was occurring. Our conclusions are based on observing the model’s output (the generated code, whether it parses correctly, and if it passes tests). We did not analyze the model’s internal workings, such as its internal thought processes (logits) or the specific details of its training data composition. Therefore, our insights are drawn from correlations in the output data.

VI. CONCLUSION

This study introduces Programming Language Confusion (PLC) as a critical and systematic failure mode in LLMs. Rather than random error, PLC reflects consistent language migrations—most often defaults to Python—driven by training biases that favor syntactic plausibility over functional correctness. Explicit language keywords and structured tasks effectively mitigate confusion, while natural language instructions have little influence.

Another critical finding for real-world application is the effect of model quantization. Quantization, a necessary step for local deployment, acts as a severe amplifier of PLC. It causes a sharp drop in language fidelity across the board and, crucially, leads to a simultaneous collapse of language adherence and syntactic quality in complex tasks like translation. This demonstrates that the high performance metrics of full-precision models are not reliable indicators of performance in resource-constrained environments.

To address these issues, development workflows must shift focus: functional correctness, not syntactic pass rates, must be the primary validation goal, as the latter is often misleading. Practitioners must use explicit technical markers in prompts to provide robust language boundaries. Finally, quantization-aware testing is essential before deployment, recognizing that practical LLM use requires rigorous verification of language stability. Future research must prioritize redesigning LLM architectures to better encode and stabilize programming language boundaries for reliable multilingual code generation.

DATA AVAILABILITY

To promote transparency and facilitate reproducibility, we make our artifacts available to the community at:

<https://github.com/TruX-DTF/PLC>

REFERENCES

- [1] J. Kaddour, J. Harris, M. Mozes, H. Bradley, R. Raileanu, and R. McHardy, "Challenges and applications of large language models," *arXiv preprint arXiv:2307.10169*, 2023.
- [2] R. Anil, A. M. Dai, O. Firat, M. Johnson, D. Lepikhin, A. Passos, S. Shakeri, E. Taropa, P. Bailey, Z. Chen *et al.*, "Palm 2 technical report," *arXiv preprint arXiv:2305.10403*, 2023.
- [3] M. Awais, M. Naseer, S. Khan, R. M. Anwer, H. Cholakkal, M. Shah, M.-H. Yang, and F. S. Khan, "Foundation models defining a new era in vision: a survey and outlook," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2025.
- [4] S. Barke, M. B. James, and N. Polikarpova, "Grounded copilot: How programmers interact with code-generating models," *Proceedings of the ACM on Programming Languages*, vol. 7, no. OOPSLA1, pp. 1–31, 2023.
- [5] C. Bird, E. Kalliamvakou, E. Murphy-Hill, and T. Zimmermann, "Taking flight with copilot: Early insights and opportunities of ai-powered pair-programming tools," in *Proceedings of the 36th IEEE/ACM International Conference on Automated Software Engineering*, 2022, pp. 1–12.
- [6] A. Sergeyuk, Y. Golubev, T. Bryksin, and I. Ahmed, "Using ai-based coding assistants in practice: State of affairs, perceptions, and ways forward," *Information and Software Technology*, vol. 178, p. 107610, 2025.
- [7] A. Ziegler, E. Kalliamvakou, Y. Chen, T. Hundhausen, A. Begel, C. Bird, and S. Bansal, "Productivity assessment of neural code completion," in *Proceedings of the 36th IEEE/ACM International Conference on Automated Software Engineering*, 2022, pp. 238–249.
- [8] F. F. Xu, U. Alon, G. Neubig, and V. J. Hellendoorn, "A systematic evaluation of large language models of code," in *Proceedings of the 6th ACM SIGPLAN International Symposium on Machine Programming*, ser. MAPS 2022. New York, NY, USA: Association for Computing Machinery, 2022, p. 1–10. [Online]. Available: <https://doi.org/10.1145/3520312.3534862>
- [9] L. Wang, W. Chen, J. Xu, S. Lei, Y. Liu, K. Zhou, Y. Zhuang, C. Chen, Y. Xie, J. Li *et al.*, "A survey on large language model based autonomous agents," *arXiv preprint arXiv:2308.11432*, 2023.
- [10] D. Nam, A. Macvean, V. Hellendoorn, B. Vasilescu, and B. Myers, "Using an llm to help with code understanding," in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, ser. ICSE '24. New York, NY, USA: Association for Computing Machinery, 2024. [Online]. Available: <https://doi.org/10.1145/3597503.3639187>
- [11] X. Du, M. Liu, K. Wang, H. Wang, J. Liu, Y. Chen, J. Feng, C. Sha, X. Peng, and Y. Lou, "Evaluating large language models in class-level code generation," in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, ser. ICSE '24. New York, NY, USA: Association for Computing Machinery, 2024. [Online]. Available: <https://doi.org/10.1145/3597503.3639219>
- [12] L. Chen, Q. Guo, H. Jia, Z. Zeng, X. Wang, Y. Xu, J. Wu, Y. Wang, Q. Gao, J. Wang *et al.*, "A survey on evaluating large language models in code generation tasks," *arXiv preprint arXiv:2408.16498*, 2024.
- [13] J. Liu, C. S. Xia, Y. Wang, and L. Zhang, "Is your code generated by chatgpt really correct? rigorous evaluation of large language models for code generation," *Advances in Neural Information Processing Systems*, vol. 36, pp. 21 558–21 572, 2023.
- [14] Z. Peng, L. Williams, and W. Ma, "The impact of ai code generators on software development practices: A mixed-methods study," in *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2023, pp. 1320–1333.
- [15] A. M. Dakhel, V. Majdinasab, A. Nikanjam, F. Khomh, and M. C. Desmarais, "Github copilot ai pair programmer: Asset or liability?" in *Proceedings of the 2023 IEEE/ACM 45th International Conference on Software Engineering*, 2023, pp. 411–423.
- [16] P. Vaithilingam, T. Zhang, and E. L. Glassman, "Expectation vs. experience: Evaluating the usability of code generation tools powered by large language models," in *Proceedings of the 35th Annual ACM Symposium on User Interface Software and Technology*, 2022, pp. 1–11.
- [17] R. Bommasani, D. A. Hudson, E. Adeli, R. Altman, S. Arora, S. von Arx, M. S. Bernstein, J. Bohg, A. Bosselut, E. Brunskill *et al.*, "On the opportunities and risks of foundation models," *arXiv preprint arXiv:2108.07258*, 2021.
- [18] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. d. O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman *et al.*, "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374*, 2021.
- [19] J. Austin, A. Odena, M. Nye, M. Bosma, H. Michalewski, D. Dohan, E. Jiang, C. J. Cai, M. Terry, Q. V. Le, and C. Sutton, "Program synthesis with large language models," *ArXiv*, vol. abs/2108.07732, 2021.
- [20] F. Liu, Y. Liu, L. Shi, H. Huang, R. Wang, Z. Yang, L. Zhang, Z. Li, and Y. Ma, "Exploring and evaluating hallucinations in llm-powered code generation," *arXiv preprint arXiv:2404.00971*, 2024.
- [21] R. Pan, A. R. Ibrahimzade, R. Krishna, D. Sankar, L. P. Wassi, M. Merler, B. Sobolev, R. Pavuluri, S. Sinha, and R. Jabbarvand, "Lost in translation: A study of bugs introduced by large language models while translating code," in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, 2024, pp. 1–13.
- [22] U. Alon, M. Zilberstein, O. Levy, and E. Yahav, "code2vec: Learning distributed representations of code," *Proceedings of the ACM on Programming Languages*, vol. 3, no. POPL, pp. 1–29, 2019.
- [23] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, D. Jiang, and M. Zhou, "CodeBERT: A pre-trained model for programming and natural languages," in *Findings of the Association for Computational Linguistics: EMNLP 2020*, T. Cohn, Y. He, and Y. Liu, Eds. Online: Association for Computational Linguistics, Nov. 2020, pp. 1536–1547. [Online]. Available: <https://aclanthology.org/2020.findings-emnlp.139/>
- [24] Y. Li, D. Choi, J. Chung, N. Kushman, J. Schrittwieser, R. Leblond, T. Eccles, J. Keeling, F. Gimeno, A. Dal Lago *et al.*, "Competition-level code generation with alphacode," *Science*, vol. 378, no. 6624, pp. 1092–1097, 2022.
- [25] B. Roziere, J. Gehring, F. Gloeckle, S. Sootla, I. Gat, X. E. Tan, Y. Adi, J. Liu, T. Remez, J. Rapin *et al.*, "Code llama: Open foundation models for code," *arXiv preprint arXiv:2308.12950*, 2023.
- [26] P. Yin and G. Neubig, "A syntactic neural model for general-purpose code generation," in *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2017, pp. 440–450.
- [27] W. U. Ahmad, S. Chakraborty, B. Ray, and K.-W. Chang, "Unified pre-training for program understanding and generation," *arXiv preprint arXiv:2103.06333*, 2021.
- [28] H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt, and R. Karri, "Asleep at the keyboard? assessing the security of github copilot's code contributions," in *2022 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2022, pp. 754–768.
- [29] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. Ponde de Oliveira Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman *et al.*, "Evaluating large language models trained on code," in *arXiv preprint arXiv:2107.03374*, 2021.
- [30] D. Sobania, M. Briesch, A. Hanna, and S. Baltes, "Choose your language model carefully: An empirical study on optimization from code completion tools," in *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*, 2022, pp. 1–12.
- [31] J. Austin, A. Odena, M. Nye, M. Bosma, H. Michalewski, D. Dohan, E. Jiang, C. Cai, M. Terry, Q. Le *et al.*, "Program synthesis with large language models," in *arXiv preprint arXiv:2108.07732*, 2021.
- [32] S. Lu, D. Guo, S. Ren, J. Huang, A. Svyatkovskiy, A. Blanco, C. Clement, D. Drain, D. Jiang, D. Tang *et al.*, "Codexglue: A machine learning benchmark dataset for code understanding and generation," in *arXiv preprint arXiv:2102.04664*, 2021.
- [33] F. Cassano, J. Gouwar, D. Nguyen, S. Nguyen, L. Phipps-Costin, D. Pinckney, M. H. Yee, Y. Zi, C. J. Anderson, J. Feldman *et al.*, "Multipl-e: A scalable and extensible approach to benchmarking neural code generation," *arXiv preprint arXiv:2208.08227*, 2022.
- [34] A. Conneau, K. Khandelwal, N. Goyal, V. Chaudhary, G. Wenzek, F. Guzmán, E. Grave, M. Ott, L. Zettlemoyer, and V. Stoyanov, "Unsupervised cross-lingual representation learning at scale," *arXiv preprint arXiv:1911.02116*, 2020.
- [35] G. I. Winata, S. Cahyawijaya, Z. Liu, Z. Lin, A. Madotto, P. Xu, and P. Fung, "Language modeling for code-switching: Evaluation, integration of monolingual data, and discriminative training," in *Proceedings*

of the 2021 Conference on Empirical Methods in Natural Language Processing, 2021, pp. 4519–4535.

- [36] Y. Wang, W. Wang, S. Joty, and S. Cardie, “Codet5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation,” in *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, 2021, pp. 8696–8708.
- [37] D. Kahneman, *Thinking, fast and slow*. Macmillan, 2011.
- [38] J. Siegmund, C. Kästner, S. Apel, C. Parnin, A. Bethmann, T. Leich, G. Saake, and A. Brechmann, “Understanding understanding source code with functional magnetic resonance imaging,” *Proceedings of the 36th International Conference on Software Engineering*, pp. 378–389, 2014.
- [39] M. J. Lee and A. J. Ko, “Knowledge transfer between programming languages,” *SIGCSE*, vol. 13, pp. 181–186, 2013.
- [40] N. Peitek, S. Apel, C. Parnin, A. Brechmann, and J. Siegmund, “Program comprehension and code complexity metrics: An fmri study,” in *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*. IEEE, 2021, pp. 524–536.
- [41] Q. Zhu, D. Guo, Z. Shao, D. Yang, P. Wang, R. Xu, Y. Wu, Y. Li, H. Gao, S. Ma *et al.*, “Deepseek-coder-v2: Breaking the barrier of closed-source models in code intelligence,” *arXiv preprint arXiv:2406.11931*, 2024.
- [42] J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altenschmidt, S. Altman, S. Anadkat *et al.*, “Gpt-4 technical report,” *arXiv preprint arXiv:2303.08774*, 2023.
- [43] OpenAI, J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altenschmidt, S. Altman, S. Anadkat, R. Avila, I. Babuschkin, S. Balaji, V. Balcom, P. Baltescu, H. Bao, M. Bavarian, J. Belgum, I. Bello, J. Berdine, G. Bernadett-Shapiro, C. Berner, L. Bogdonoff, O. Boiko, M. Boyd, A.-L. Brakman, G. Brockman, T. Brooks, M. Brundage, K. Button, T. Cai, R. Campbell, A. Cann, B. Carey, C. Carlson, R. Carmichael, B. Chan, C. Chang, F. Chantzis, D. Chen, S. Chen, R. Chen, J. Chen, M. Chen, B. Chess, C. Cho, C. Chu, H. W. Chung, D. Cummings, J. Currier, Y. Dai, C. Decareaux, T. Degry, N. Deutsch, D. Deville, A. Dhar, D. Dohan, S. Dowling, S. Dunning, A. Ecoffet, A. Eleti, T. Eloundou, D. Farhi, L. Fedus, N. Felix, S. P. Fishman, J. Forte, I. Fulford, L. Gao, E. Georges, C. Gibson, V. Goel, T. Gogineni, G. Goh, R. Gontijo-Lopes, J. Gordon, M. Grafstein, S. Gray, R. Greene, J. Gross, S. S. Gu, Y. Guo, C. Hallacy, J. Han, J. Harris, Y. He, M. Heaton, J. Heidecke, C. Hesse, A. Hickey, W. Hickey, P. Hoeschele, B. Houghton, K. Hsu, S. Hu, X. Hu, J. Huizinga, S. Jain, S. Jain, J. Jang, A. Jiang, R. Jiang, H. Jin, D. Jin, S. Jomoto, B. Jonn, H. Jun, T. Kaftan, Łukasz Kaiser, A. Kamali, I. Kanitscheider, N. S. Keskar, T. Khan, L. Kilpatrick, J. W. Kim, C. Kim, Y. Kim, J. H. Kirchner, J. Kiros, M. Knight, D. Kokotajlo, Łukasz Kondraciuk, A. Kondrich, A. Konstantinidis, K. Kosic, G. Krueger, V. Kuo, M. Lampe, I. Lan, T. Lee, J. Leike, J. Leung, D. Levy, C. M. Li, R. Lim, M. Lin, S. Lin, M. Litwin, T. Lopez, R. Lowe, P. Lue, A. Makanju, K. Malfacini, S. Manning, T. Markov, Y. Markovski, B. Martin, K. Mayer, A. Mayne, B. McGrew, S. M. McKinney, C. McLeavey, P. McMillan, J. McNeil, D. Medina, A. Mehta, J. Menick, L. Metz, A. Mishchenko, P. Mishkin, V. Monaco, E. Morikawa, D. Mossing, T. Mu, M. Murati, O. Murk, D. Mély, A. Nair, R. Nakano, R. Nayak, A. Neelakantan, R. Ngo, H. Noh, L. Ouyang, C. O’Keefe, J. Pachocki, A. Paino, J. Palermo, A. Pantuliano, G. Parascandolo, J. Parish, E. Parparita, A. Passos, M. Pavlov, A. Peng, A. Perelman, F. de Avila Belbute Peres, M. Petrov, H. P. de Oliveira Pinto, Michael, Pokorny, M. Pokrass, V. H. Pong, T. Powell, A. Power, B. Power, E. Proehl, R. Puri, A. Radford, J. Rae, A. Ramesh, C. Raymond, F. Real, K. Rimbach, C. Ross, B. Rotsted, H. Roussez, N. Ryder, M. Saltarelli, T. Sanders, S. Santurkar, G. Sastry, H. Schmidt, D. Schnurr, J. Schulman, D. Selsam, K. Sheppard, T. Sherbakov, J. Shieh, S. Shoker, P. Shyam, S. Sidor, E. Sigler, M. Simens, J. Sitkin, K. Slama, I. Sohl, B. Sokolowsky, Y. Song, N. Staudacher, F. P. Such, N. Summers, I. Sutskever, J. Tang, N. Tezak, M. B. Thompson, P. Tillett, A. Tootoonchian, E. Tseng, P. Tuggle, N. Turley, J. Tworek, J. F. C. Uribe, A. Vallone, A. Vijayvergiya, C. Voss, C. Wainwright, J. J. Wang, A. Wang, B. Wang, J. Ward, J. Wei, C. Weinmann, A. Welihinda, P. Welinder, J. Weng, L. Weng, M. Wiethoff, D. Willner, C. Winter, S. Wolrich, H. Wong, L. Workman, S. Wu, J. Wu, M. Wu, K. Xiao, T. Xu, S. Yoo, K. Yu, Q. Yuan, W. Zaremba, R. Zellers, C. Zhang, M. Zhang, S. Zhao, T. Zheng, J. Zhuang, W. Zhuk, and B. Zoph, “Gpt-4 technical report,” 2024. [Online]. Available: <https://arxiv.org/abs/2303.08774>
- [44] A. Grattafiori, A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Vaughan *et al.*, “The llama 3 herd of models,” *arXiv preprint arXiv:2407.21783*, 2024.
- [45] A. Jiang, A. Sablayrolles, A. Mensch, C. Bamford, D. Chaplot, D. de Las Casas, F. Bressand, G. Lengyel, G. Lample, L. Saulnier *et al.*, “Mistral 7b. corr, abs/2310.06825, 2023. doi: 10.48550,” *arXiv preprint ARXIV.2310.06825*.
- [46] Q. Team, “Qwen2.5: A party of foundation models,” September 2024. [Online]. Available: <https://qwenlm.github.io/blog/qwen2.5/>
- [47] Y. Wei, F. Cassano, J. Liu, Y. Ding, N. Jain, Z. Mueller, H. de Vries, L. von Werra, A. Guha, and L. Zhang, “Selfcodealign: Self-alignment for code generation,” *arXiv preprint arXiv:2410.24198*, 2024.
- [48] N. Jain, K. Han, A. Gu, W.-D. Li, F. Yan, T. Zhang, S. Wang, A. Solar-Lezama, K. Sen, and I. Stoica, “Livecodebench: Holistic and contamination free evaluation of large language models for code,” *arXiv preprint arXiv:2403.07974*, 2024.
- [49] G. Orlanski, K. Xiao, X. Garcia, J. Hui, J. Howland, J. Malmaud, J. Austin, R. Singh, and M. Catasta, “Measuring the impact of programming language distribution,” in *International Conference on Machine Learning*. PMLR, 2023, pp. 26 619–26 645.
- [50] T. Schuster, A. Kalyan, A. Polozov, and A. T. Kalai, “Programming puzzles,” in *Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2021. [Online]. Available: <https://arxiv.org/abs/2106.05784>
- [51] Q. Peng, Y. Chai, and X. Li, “HumanEval-XL: A multilingual code generation benchmark for cross-lingual natural language generalization,” in *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024)*, N. Calzolari, M.-Y. Kan, V. Hoste, A. Lenci, S. Sakti, and N. Xue, Eds. Torino, Italia: ELRA and ICCL, May 2024, pp. 8383–8394. [Online]. Available: <https://aclanthology.org/2024.lrec-main.735/>
- [52] L. Chai, S. Liu, J. Yang, Y. Yin, K. Jin, J. Liu, T. Sun, G. Zhang, C. Ren, H. Guo, Z. Wang, B. Wang, X. Wu, B. Wang, T. Li, L. Yang, S. Duan, and Z. Li, “Mceval: Massively multilingual code evaluation,” 2024. [Online]. Available: <https://arxiv.org/abs/2406.07436>
- [53] O. Rodriguez-Prieto, A. Pato, and F. Ortin, “Plangrec: Deep-learning model to predict the programming language from a single line of code,” *Future Generation Computer Systems*, vol. 166, p. 107640, 2025. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0167739X24006046>